

SOP 21 — Write Cards

Camada A.N.I.: Instruments-Tools (write_cards.py) · Versão: 1.0 (Protocol 0, 2026-09-09)

Golden Rule: mudou a lógica → atualiza o SOP → só então o código.

Objetivo

Persistir os `ProcessedOutput.NewsCard[]` validados (SOP 20) como registro **durável** do dia: primeiro na bancada `.tmp/`, e na cloud (aba `news_cards`) quando o Google Sheets/Supabase estiver conectado. É a “escrita” entre classificar e montar o payload — sem lógica editorial nova, sem LLM.

Quando roda

- Imediatamente após `classify_and_dedupe` no cron daily build 07:15 e nos refreshes 12:00/18:00.
- Nunca roda sozinho fora desses fluxos (a não ser em teste).

Input

- `ProcessedOutput.NewsCard[]` de `.tmp/cards/<YYYY-MM-DD>.json` (saída do SOP 20).
- Apenas cards válidos contra o schema (inválidos não chegam aqui).

Output

- `.tmp/cards_final/<YYYY-MM-DD>.json` — cards prontos para consumo do SOP 30 (espelho do que vai para a cloud).
- Cloud (quando disponível): aba `news_cards` — uma linha por `card_id`; escrita idempotente (upsert por `card_id`).
- JSON de relatório: `{ "cards_written": n, "cloud": "ok|pending" }`.

Passos determinísticos

1. Ler o lote de cards validados.
2. Conferência de integridade (sem LLM): todo card tem `card_id` único, `stamp` em enum, `category` em enum, `source_url` preenchido, `source_name` preenchido; `duplicate_of` aponta para `card_id` existente no mesmo lote ou em lote anterior do dia.
3. Rejeitar card com qualquer campo obrigatório ausente → devolver ao SOP 20 como `needs_review` (log em `progress.md`).
4. Gravar `.tmp/cards_final/<YYYY-MM-DD>.json`.
5. Se a cloud estiver acessível: upsert na aba `news_cards` (chave: `card_id`); senão, marcar `cloud: "pending"` no relatório — **nunca** escrever na cloud a partir de `.tmp/` sem validação, e nunca tratar `.tmp/` como verdade.
6. Reportar resumo.

Edge cases

- `card_id` duplicado no lote → manter o primeiro e logar (os IDs são gerados no SOP 20; duplicidade indica bug → SOP 90).
- Cloud indisponível na escrita → `cloud: "pending"` ; o lote fica na bancada e a sincronização é tentada no próximo ciclo (12:00/18:00). Payload não é publicado sem passar pela cloud (regra BLAST).
- Card com `needs_review: true` → **é** gravado na bancada (para auditoria) mas fica sinalizado; não entra no payload aprovado sem olho humano.
- Enum fora da CONSTITUTION (ex.: `stamp` inventado) → rejeição na conferência de integridade, nunca correção silenciosa.

Rate limits / limites conhecidos

- Os da planilha/banco escolhido (gravações em lote com backoff em erro 429/5xx; fixar valores reais na Phase L).

Testes

- Fixture: rodar sobre a saída esperada de `fixtures/news_bundle.json` processada (bundle → cards → gravação idempotente 2x no mesmo `card_id` não duplica linha).
- Phase L: `tools/ping_sheets.py` valida o caminho de escrita na aba `news_cards` .